Indicaciones:

1. De acuerdo con los siguientes casos reales, analice y justifique qué método de ordenamiento utilizaría (InsertionSort, BubbleSort, SelectionSort o QuickSort):
    a. **Caso 1: Biblioteca pequeña**: Una biblioteca de un colegio tiene solo 20 libros y cada semana se agregan algunos nuevos.

    b. **Caso 2: Supermercado:** Un supermercado quiere ordenar los precios de miles de productos para mostrar las ofertas en pantalla.

    c. **Caso 3: Inmobiliaria:** Una inmobiliaria quiere ordenar una base de datos con miles de casas por precio de venta.

    d. **Caso 4: Taller mecánico:** Un taller mecánico recibe una lista de 50 autos y quiere ordenarlos por año de fabricación.

2. Tome uno de los ejemplos y desarrolle el código de manera sencilla.
3. Investigue sobre cómo funciona el MergeSort y realice un ejemplo programado.

1. **Biblioteca pequeña: InsertionSort.** Es adecuado para 20 libros y funciona bien cuando la lista ya está ordenada y se agregan pocos libros. Cada libro nuevo se inserta en su posición. Su tiempo en el peor caso es ***O(n^2).***

2. **Supermercado: QuickSort.** Es apropiado para ordenar miles de precios por su tiempo promedio de ***O(n \log n).*** En el peor caso tarda ***O(n^2)***, por lo que conviene elegir el pivote cuidadosamente.

3. **Inmobiliaria: QuickSort.** También es útil para ordenar miles de casas por precio, gracias a su tiempo promedio de ***O(n \log n).*** Si es importante conservar el orden original de las casas con el mismo precio, MergeSort es una alternativa estable.

4. **Taller mecánico: InsertionSort.** Para 50 autos es sencillo y suficientemente práctico. Además, conserva el orden original de los autos que tienen el mismo año.

BubbleSort y SelectionSort son menos convenientes para las listas grandes porque tienen una complejidad promedio de ***O(n^2)***.

Ejemplo en Python: InsertionSort para el taller:

```python
autos = [
    {"nombre": "Auto A", "anio": 2018},
    {"nombre": "Auto B", "anio": 2012},
    {"nombre": "Auto C", "anio": 2021},
    {"nombre": "Auto D", "anio": 2015}
]

for i in range(1, len(autos)):
    actual = autos[i]
    j = i - 1

    while j >= 0 and autos[j]["anio"] > actual["anio"]:
        autos[j + 1] = autos[j]
        j -= 1

    autos[j + 1] = actual

for auto in autos:
    print(auto["nombre"], auto["anio"])
```

Resultado:

```
Auto B 2012
Auto D 2015
Auto A 2018
Auto C 2021
```

MergeSort

MergeSort aplica la estrategia de ***divide y vencerás***:

1. Divide la lista en dos mitades hasta obtener listas de un elemento.
2. Combina las listas comparando sus elementos para formar listas ordenadas.
3. Repite la combinación hasta obtener la lista completa ordenada.

Su complejidad temporal es ***O(n \log n)*** incluso en el peor caso. Es estable, aunque generalmente necesita espacio adicional de ***O(n)***.

```python
def merge_sort(lista):
    if len(lista) <= 1:
        return lista

    mitad = len(lista) // 2
    izquierda = merge_sort(lista[:mitad])
    derecha = merge_sort(lista[mitad:])

    resultado = []
    i = 0
    j = 0

    while i < len(izquierda) and j < len(derecha):
        if izquierda[i] <= derecha[j]:
            resultado.append(izquierda[i])
            i += 1
        else:
            resultado.append(derecha[j])
            j += 1

    resultado.extend(izquierda[i:])
    resultado.extend(derecha[j:])
    return resultado


precios = [2500, 1200, 3800, 900, 1700]
print(merge_sort(precios))
```

Resultado:

```
[900, 1200, 1700, 2500, 3800]
```